

Basis

线性基

回想高中数学立体几何中基向量的概念，我们可以在三维欧氏空间中找到一组基向量，，，之后空间中任意一个向量都可以由这组基向量表示。换句话说，我们可以 **通过有限的基向量来描述无限的三维空间**，这足以体现基向量的重要性。

三维欧氏空间是特殊的 [线性空间](#)，三维欧氏空间的基向量在线性空间中就被推广为了线性基。

OI 中有关线性基的应用一般只涉及两类线性空间：维实线性空间和维 [布尔域](#) 线性空间，我们会在 [应用](#) 一节中详细介绍。若您不熟悉线性代数，则推荐从应用部分开始阅读。

以下会从一般的线性空间出发来介绍线性基，并给出线性基的常见性质。

前置知识：[线性空间](#)。

线性基是线性空间的一组基，是研究线性空间的重要工具。

定义

称线性空间的一个极大线性无关组为的一组 **Hamel 基** 或 **线性基**，简称 **基**。

规定线性空间的基为空集。

可以证明任意线性空间均存在线性基¹，我们定义线性空间的 **维数** 为线性基的元素个数（或势），记作。

性质

1. 对于有限维线性空间，设其维数为，则：

- 中的任意个向量线性相关。
- 中的任意个线性无关的向量均为的基。
- 若中的任意向量均可被向量组线性表出，则其是的的一个基。

▼ 证明

任取中的一组基，由已知条件，向量组可被线性表出，故

因此

- 中任意线性无关向量组均可通过插入一些向量使得其变为的一个基。

2. （子空间维数公式）令是关于的有限维线性空间，且和也是有限维的，则

▼ 证明

设，

取的一组基，将其分别扩充为和中的基：和。

接下来只需证明向量组 线性无关即可。

设 .

则 .

注意到上式左边在 中，右边在 中，故两边均在 中，因此

故，进而

3. 令 是关于 的有限维线性空间，且 和 也是有限维的，则下列诸款等价：

- 1..
- 2..
- 3. 若 是 的一组基， 是 的一组基，则 是 的一组基。

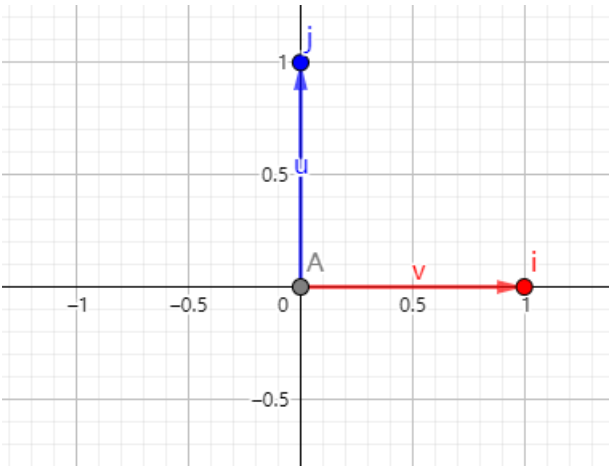
▼ Note

1,3 两条可推广到无限维线性空间中

例子

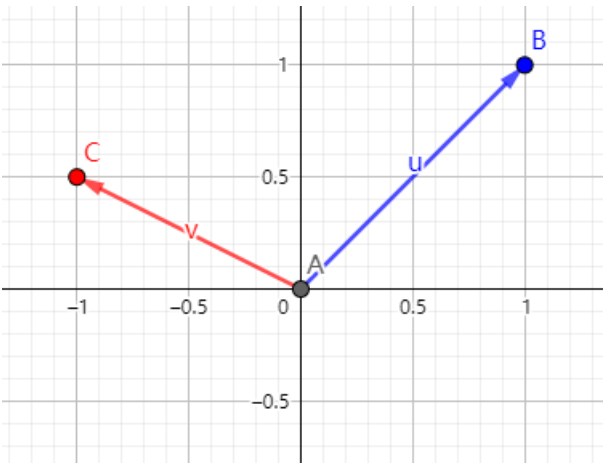
考虑 的基。

1. 如图



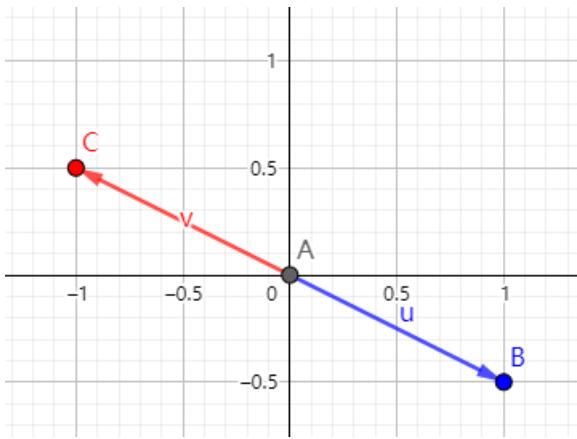
是一组基。

2. 如图



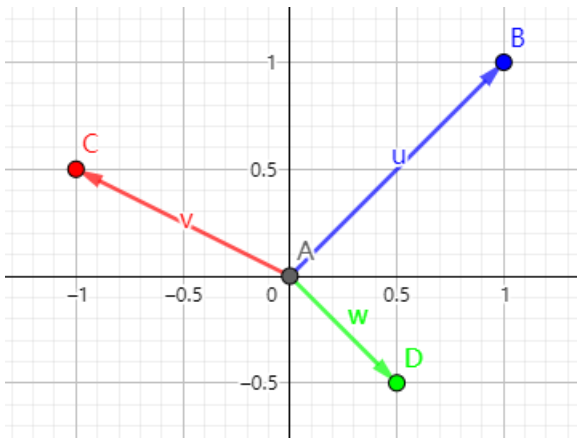
是一组基。

3. 如图



不是一组基，因为 .

4. 如图



不是一组基，因为 .

正交基与单位正交基

若线性空间 的一组基 满足（即两两正交），则称这组基是 **正交基**。

若线性空间 的一组正交基 还满足，则称这组基是 **单位正交基**。

任意有限维线性空间 的基都可以通过 [Schmidt 正交化](#) 变换为正交基。

应用

根据前文内容，我们可以利用线性基实现：

1. 求给定向量组的秩；
2. 对给定的向量组，找到一组极大线性无关组（或其张成的线性空间的一组基）；
3. 向给定的向量组插入某些向量，在插入操作后的向量组中找到一组极大线性无关组（或其张成的线性空间的一组基）；
4. 对找到的一组极大线性无关组（或基），判断某向量能否被其线性表出；
5. 对找到的一组极大线性无关组（或基），求其张成的线性空间中的特殊元素（如最大元、最小元等）。

在 OI 中，我们一般将 维实线性空间 下的线性基称为 **实数线性基**， 维布尔域线性空间 下的线性基称为 **异或线性基**。

▼ Tip

中的加法为异或，乘法为与，可以证明 $\mathbb{F}_2^n$ 是域。

可以证明代数系统 $(\mathbb{F}_2^n, +, \cdot)$ 是线性空间，其中：

即加法是异或，数乘是与。

以异或线性基为例，我们可以根据给定的一组布尔序列 构造出一组异或线性基，这组基有如下性质：

1. $\mathbb{F}_2^n$ 中任意非空子集的异或和不为 0；
2. 对 $\mathbb{F}_2^n$ 中的任意元素 x ，都可在 基中取出若干元素使其异或和为 x ；
3. 对任意满足上两条的集合 B ，其元素个数不会小于 基的元素个数。

我们可以利用异或线性基实现：

1. 判断一个数能否表示成某数集子集的异或和；
2. 求一个数表示成某数集子集异或和的方案数；
3. 求某数集子集的最大/最小/第 k 大/第 k 小异或和；
4. 求一个数在某数集子集异或和中的排名。

构造方法

因为异或线性基与实数线性基没有本质差别，所以接下来以异或线性基为例，实数线性基版本的代码只需做一点简单修改即可。

贪心法

对原集合的每个数 转为二进制，从高位向低位扫，对于第 i 位是 1 的，如果 基中不存在，那么令 e_i 并结束扫描，如果存在，令 $x = x \oplus e_i$ 。

查询原集合内任意几个元素 的最大值，只需将线性基从高位向低位扫，若 e_i 上当前扫到的 答案变大，就把答案异或上 e_i 。

为什么能行呢？因为从高往低位扫，若当前扫到第 i 位，意味着可以保证答案的第 i 位为 1，且后面没有机会改变第 i 位。

查询原集合内任意几个元素 的最小值，就是线性基集合所有元素中最小的那个。

查询某个数是否能被异或出来，类似于插入，如果最后插入的数 被异或成了 0，则能被异或出来。

► 代码（洛谷 P3812 [【模板】线性基](#)）

高斯消元法

高斯消元法相当于从线性方程组的角度去构造线性基，正确性显然。

► 代码（洛谷 P3812 [【模板】线性基](#)）

性质

贪心法构造的线性基具有如下性质：

- 线性基没有异或和为 0 的子集。
- 线性基中各数二进制最高位不同。

高斯消元法构造出的线性基满足如下性质：

- 高斯消元后的矩阵是一个行简化阶梯形矩阵。

该性质包含了贪心法构造的线性基满足的两条性质

如果不理解这条性质的正确性，可以跳转 [高斯消元](#)。

提供一组样例：

```
1 5
2 633 211 169 841 1008
```

二进制表示：

```
1 1001111001
2 0011010011
3 0010101001
4 1101001001
5 1111110000
```

贪心法生成的线性基：

```
1 1001111001
2 0100110000
3 0011010011
4 0001111010
5 0000000000
6 0000010000
7 0000000000
8 0000000000
9 0000000000
10 0000000000
```

高斯消元法生成的线性基：

```
1 1000000011
2 0100110000
3 0010101001
4 0001101010
5 0000010000
6 0000000000
7 0000000000
8 0000000000
9 0000000000
10 0000000000
```

这是一条非常好的性质，能帮我们更方便的解决很多问题。比如：给定一些数，选其中一些异或起来，求异或最大值，如果用贪心法构造线性基，需要再做一遍贪心，如果 `ans` 的当前位是 0，那么异或一定会更优，否则当前位如果为 1，则一定不会更优；而使用高斯消元法构造线性基后直接将线性基中所有元素都异或起来输出即可。

对于其他比较经典的问题（查询一个数能否被异或得到，查询能被异或得到的第 `k` 大数等），高斯消元法得到的线性基也能更加方便地解决。

时间复杂度

设向量长度为 `n`，总数为 `m`，则时间复杂度为 $\mathcal{O}(nm)$ 。其中高斯消元法的常数略大。

若是实数线性基，则时间复杂度为 $\mathcal{O}(nm^2)$ 。

线性基合并

线性基的合并只需要暴力处理，即将要合并的一组线性基暴力地插入到另一组线性基即可。单次合并的时间复杂度是 $\mathcal{O}(n \log V)$ （异或线性基）或 $\mathcal{O}(n \log V)$ （实数线性基）。

线性基求交

线性基求交，严格地说就是求它们张成的两个线性空间的交空间的一组线性基。本节介绍两种算法。这两种算法，单次求交的时间复杂度都是 $\mathcal{O}(n \log V)$ （异或线性基）或 $\mathcal{O}(n \log V)$ （实数线性基）。

朴素算法

设要求交的线性基分别是 B_1 和 B_2 。线性基求交的算法只需要对线性基暴力合并的算法做如下调整：（以异或线性基为例）

- 将线性基 B_1 中的向量 v 利用 [贪心法](#) 尝试插入到 B_2 中，并初始化线性基的交 I 为空集；
- 在插入时，需要记录要插入的向量中，线性基 B_2 中元素的贡献。具体地，维持一个新向量 u ，初始化为 v ，而且，如果正在插入的向量与线性基中第 i 位的向量取了异或，那么贡献 u_i 也要与第 i 位记录的贡献 u_i 取一次异或；
- 如果插入成功，在线性基的第 i 位插入了向量 u ，就将第 i 位记录的 u_i 改为得到 u 的过程中线性基 B_2 中元素的贡献；
- 如果插入不成功，就将过程中记录到的线性基 B_2 中元素的贡献 u_i 插入到 I 中。

这样得到的线性基 I 就是所求的交，当然，该算法同时也求出了线性基的并。

► 对算法的解释

模板题代码如下：

► 代码 (Library Checker [Intersection of vector spaces](#))

Zassenhaus 算法

另一种等价的做法是 Zassenhaus 算法，它同样可以同时计算出两个线性基的并和交。复杂度和上文完全一致。

具体步骤如下：

- 初始化一个向量长度为 V 的线性基 B 为空，其中的向量写成 (v, c) 的形式，且 v 和 c 长度均为 V ；
- 将 B_1 中的元素 (v, c) 以 $(v, c \oplus c')$ 的形式插入 B 中；
- 将 B_2 中的元素 (v, c) 以 $(v, c \oplus c')$ 的形式插入 B 中；
- 最后得到的线性基 B 中的所有非零元素 (v, c) 中，非零的那些向量中项 v 的全体组成了 B_1 和 B_2 的并的线性基，为零的那些向量中项 v 的全体组成了 B_1 和 B_2 的交的线性基。

算法中的构造线性基的方法可以是 [贪心法](#) 或 [高斯消元法](#)，只要保证 B 中的线性基组成行阶梯型矩阵即可。

将 Zassenhaus 算法中的消元的步骤与上面的朴素算法相比较，很容易发现，基于贪心法的 Zassenhaus 算法相当于维护 B 中元素的贡献的朴素算法。如果转而先插入所有 B_1 ，再插入所有 B_2 ，那么基于贪心法的 Zassenhaus 算法就相当于维护 B 中元素贡献的朴素算法。根据消元步骤的等价性，Zassenhaus 算法的正确性也是成立的。

除此之外，还可以再提供一个独立且更为一般的代数证明：

► 正确性证明

模板题代码如下：

► 代码 (Library Checker [Intersection of vector spaces](#))

注意，输出时只需要考虑前 V 位均为零的向量即可。

拓展：前缀线性基

本节只讨论异或线性基的情形，并假设单个向量可以存储在 的空间内，且单次操作复杂度总是 的。

对于需要多次查询区间异或最大值的情形，一种常见的做法是 [猫树](#) 配合线性基，时间复杂度为 $O(n \log n \log m)$ ，其中， n 是向量长度， m 是序列长度， q 是询问次数。另一种可行的做法是利用前缀线性基（或称时间戳线性基），可以将复杂度降低到 $O(n \log m)$ 。

前缀线性基允许对于序列的每个前缀，都维护该前缀的所有后缀的线性基，这样就可以支持查询每个区间的线性基。注意到序列的某个前缀 的所有后缀 的线性基是相互包含的，即 的线性基总是包含着 的线性基，所以，这些后缀的线性基中互不相同的至多只有 $\log m$ 种，而且总是可以通过向空集中逐步添加新的向量来得到自 到 所有这些后缀的线性基。因此，利用这个单调性，只需要为添加的每个向量，都标记它出现的最大下标，就可以在 的空间内存储所有后缀的线性基。而且，查询区间 对应的线性基时，只需要在 处的前缀线性基中仅保留标记 的那些向量即可。

不妨将每个向量 的标记 称为它的时间戳。线性基中的向量 总是可以表示为原序列中某些元素的异或和，比如 v_i 。而在所有这样的可能的表示中，最小下标的最大值就是 pos_i ，即

这个表达式不过是将上一段的叙述用形式的语言写出来而已。它给我们带来的启发是，要维护线性基中每个向量 的时间戳，只需要贪心地选取尽可能新的向量替换掉旧的向量即可。

基于上文提到的 [贪心法](#) 构造线性基，前缀线性基在构造过程中做了如下调整：

- 为线性基中保留的每个向量 都保存一个时间戳，初始时均设为 0；
- 要添加序列中第 i 个向量，仍然从高位向低位扫，但同时需要记录当前时间；
- 如果 的第 j 位是一，就比较线性基中已有的向量 的时间戳 和当前时间：
 - 如果 $pos_j < i$ ，即要添加的向量时间更晚，就将 v_j 设为 v_i ，并更新时间戳为 i ，并将旧的 异或 的结果 按照之前记录的时间 继续添加过程；
 - 如果 $pos_j \geq i$ ，即要添加的向量时间更早，不更新 v_j 和 pos_j ，将 v_i 异或 v_j 后继续添加即可。

也就是说，如果当前位可以通过较新的向量表示，就直接用较新的向量；否则，保留原来的向量。在更新位置 的向量时，不能将异或的结果 存入位置 j ，因为异或的结果 的时间戳为 i ，小于要添加的变量 的时间戳 i 。同样的原因，[高斯消元法](#) 构造线性基的过程中向上更新时可能会破坏时间戳的性质，所以不再适用于构造前缀线性基。

模板题代码如下：

► 代码 (Codeforces [1100F Ivan and Burgers](#))

如果需要在线询问，也可以用 的空间将每个前缀处的前缀线性基都存下来再查询，这可以看作是一种「可持久化」线性基。如果需要用到高斯消元法得到的线性基的性质，可以在查询时另行处理。

练习题

- [Luogu P3812 【模板】线性基](#)
- [Acwing 3164. 线性基](#)
- [SGU 275 to xor or not xor](#)
- [HDU 3949 XOR](#)
- [HDU 6579 Operation](#)
- [Luogu P4151 \[WC2011\] 最大 XOR 和路径](#)
- [Library Checker - Intersection of vector spaces](#)
- [AtCoder Grand Contest 045 A - Xor Battle](#)
- [Codeforces 1100F Ivan and Burgers](#)
- [Luogu P3292 \[SCOI2016\] 幸运数字](#)

参考资料与注释

1. 丘维声，高等代数（下）。清华大学出版社。

2. [Basis \(linear algebra\) - Wikipedia](#)
 3. [Vector Basis -- from Wolfram MathWorld](#)
 4. [Zassenhaus algorithm - Wikipedia](#)
-

1. [Proof that every vector space has a basis ↩](#)
-

本页面最近更新：2025/4/7 00:24:17, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cesonic](#), [Enter-tainer](#), [Great-designer](#), [lr1d](#), [ksyx](#), [lychees](#), [MegaOwler](#), [RUIN-RISE](#), [Tiphereth-A](#), [wjy-yy](#), [aofall](#), [c-forrest](#), [CoelacanthusHex](#), [iamtwz](#), [Marcythm](#), [Menci](#), [ouuan](#), [Persdre](#), [rsdbkhsky](#), [shuzhouliu](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用